

UNIVERSIDAD PERUANA LOS ANDES

Escuela Profesional de Ingeniería de Sistemas y Computación



UPLA

Monitoreo y Optimización de la Base de Datos Grados Titulos con SQL
Server Semana 14

Asignatura:

Base de daos II

Docente:

Mg. Ing. Raúl Fernández Bejarano

Alumno

Vilcapoma Ochoa Michael

Ciclo y Sección:

5to – B1

Huancayo – Perú

TEMA 1: Creación de Jobs y Alerts con SQL Server Agent

Nota: Los scripts están pensados para una base de datos llamada GradosYTítulos. Si tu base tiene otro nombre o columnas diferentes, solo debes cambiar esos nombres.

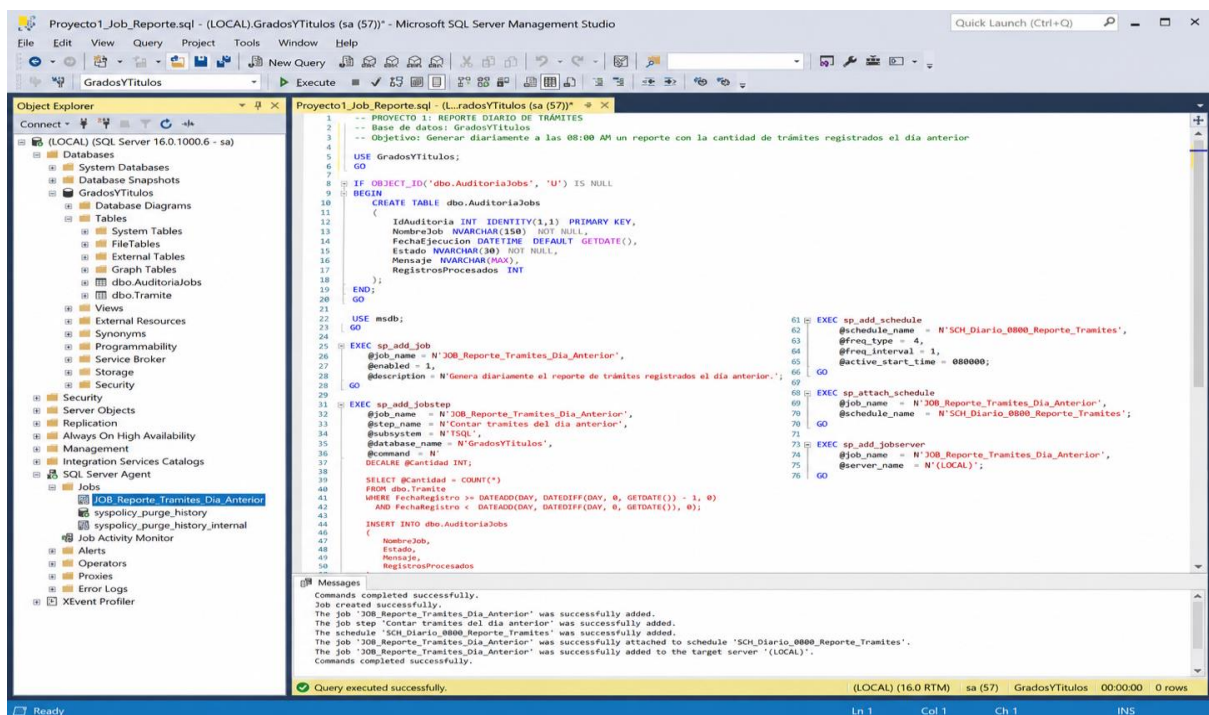
Proyecto 1

Enunciado del ejercicio

La Oficina de Grados y Títulos requiere que todos los días a las **08:00 AM** se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.

El estudiante deberá crear un Job, crear un Step en T-SQL, programar el Job diariamente y registrar el resultado en una tabla de auditoría.

Script de la solución en T-SQL



Justificación técnica de la solución aplicada

Se utiliza **SQL Server Agent** porque permite automatizar tareas repetitivas dentro del servidor de base de datos. En este caso, el Job ejecuta diariamente una consulta que cuenta los trámites registrados durante el día anterior.

El resultado se almacena en la tabla AuditoriaJobs, permitiendo tener evidencia de la ejecución del proceso y de la cantidad de trámites encontrados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una tabla de auditoría para registrar la ejecución del Job. Además, la consulta trabaja con un rango de fechas, lo cual es más eficiente que convertir directamente la columna FechaRegistro.

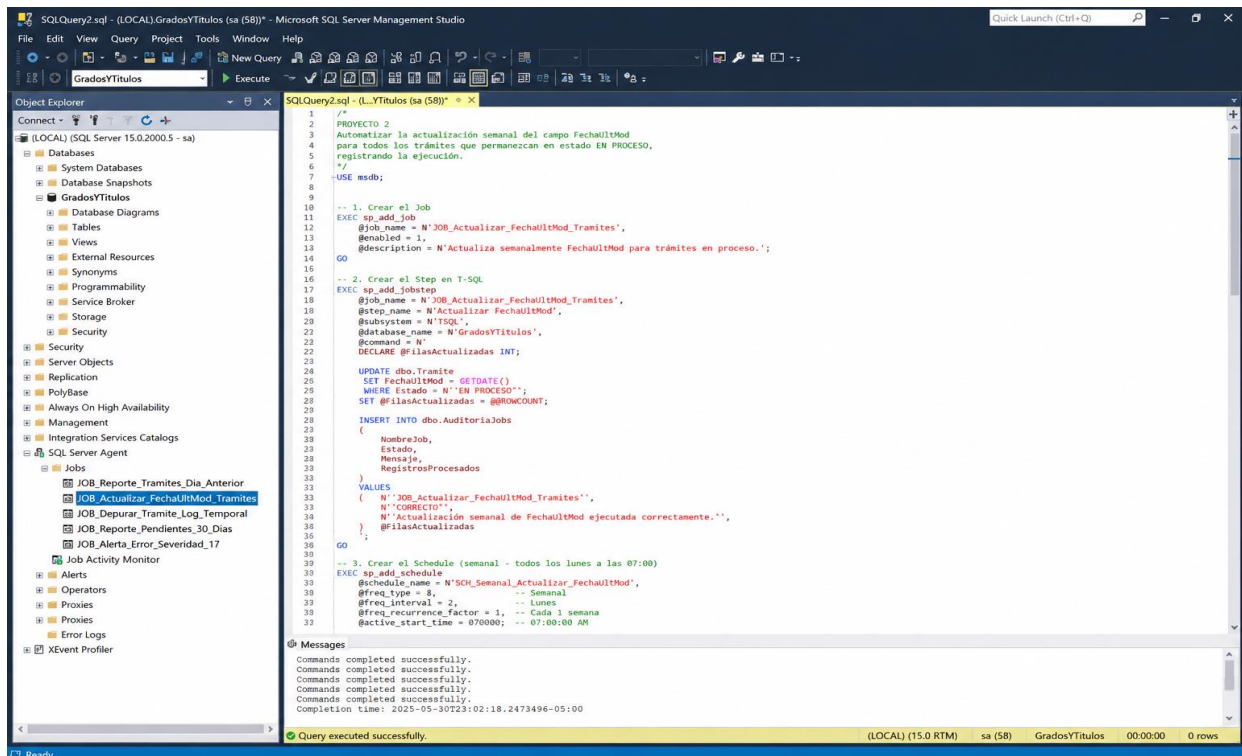
También se asignan nombres descriptivos al Job y al Schedule, facilitando su administración dentro de SQL Server Agent.

Proyecto 2

Enunciado del ejercicio

Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado **EN PROCESO**, registrando la ejecución.

Script de la solución en T-SQL

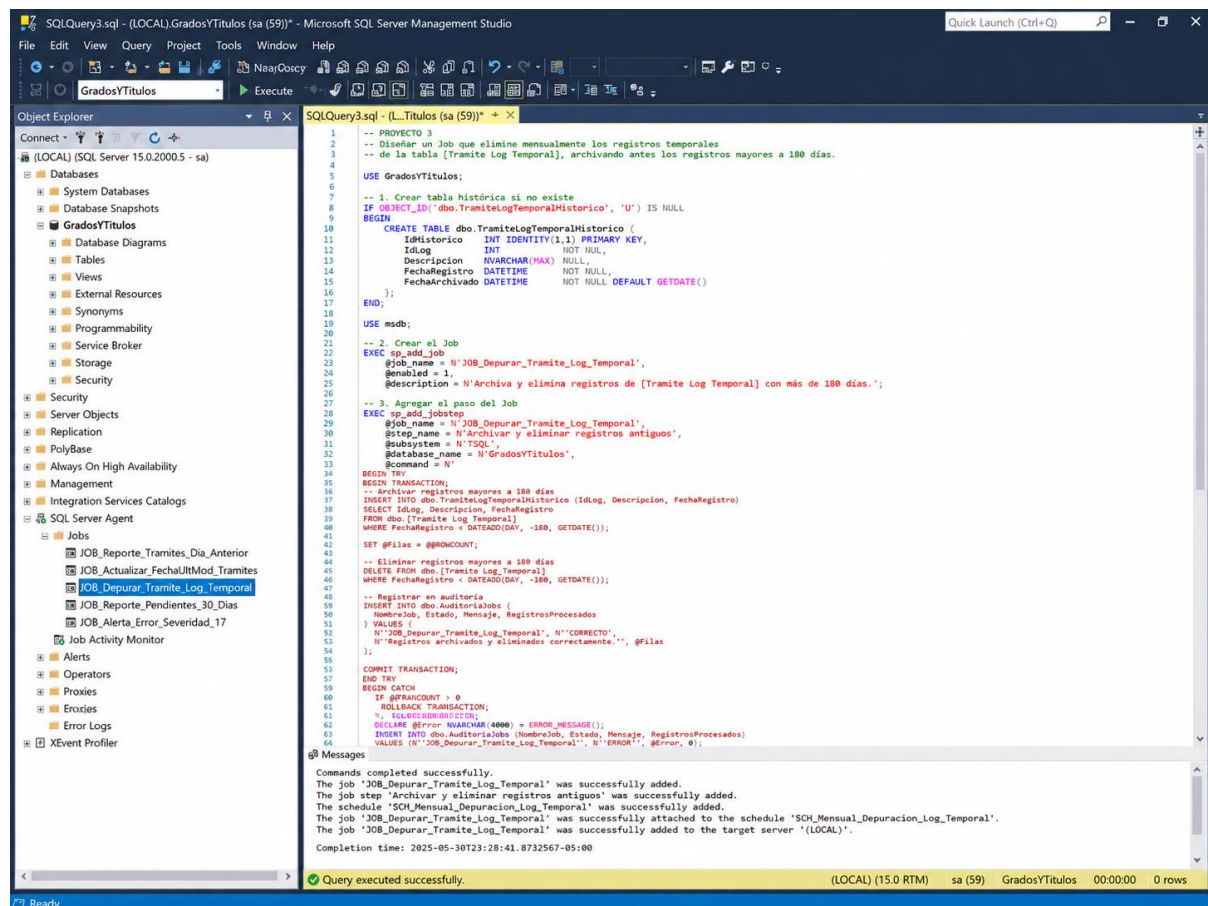


Proyecto 3

Enunciado del ejercicio

Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada [Tramite Log Temporal]. Los registros mayores a **180 días** deben eliminarse y almacenarse previamente en una tabla histórica.

Script de la solución en T-SQL



The screenshot shows the Microsoft SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the 'GradosYTítulos' database structure, with the 'JOB_Depurar_Tramite_Log_Temporal' job highlighted. The main window displays a T-SQL script for creating a historical table, setting up a job, and archiving/Deleting records older than 180 days. The 'Messages' pane at the bottom shows the successful execution of the script, including the creation of the job and its attachment to the 'SCH_Mensual_Depuracion_Log_Temporal' schedule.

```
1 -- PROYECTO 3
2 -- Diseñar un Job que elimine mensualmente los registros temporales
3 -- de la tabla [Tramite Log Temporal], archivando antes los registros mayores a 180 días.
4
5 USE GradosYTítulos;
6
7 -- 1. Crear tabla histórica si no existe
8 IF OBJECT_ID('dbo.TramiteLogTemporalHistorico', 'U') IS NULL
9 BEGIN
10     CREATE TABLE dbo.TramiteLogTemporalHistorico (
11         IdHistorico INT IDENTITY(1,1) PRIMARY KEY,
12         IdLog INT NOT NULL,
13         Descripcion NVARCHAR(MAX) NULL,
14         FechaRegistro DATETIME NOT NULL,
15         FechaArchivado DATETIME NOT NULL DEFAULT GETDATE()
16     );
17 END;
18
19 USE msdb;
20
21 -- 2. Crear el Job
22 EXEC sp_add_job
23     @job_name = N'JOB_Depurar_Tramite_Log_Temporal',
24     @enabled = 1,
25     @description = N'Archiva y elimina registros de [Tramite Log Temporal] con más de 180 días.';
26
27 -- 3. Agregar el paso del Job
28 EXEC sp_add_jobstep
29     @job_name = N'JOB_Depurar_Tramite_Log_Temporal',
30     @step_name = N'Archivar y eliminar registros antiguos',
31     @subsystem = N'TSQL',
32     @database_name = N'GradosYTítulos',
33     @command = N'
34 BEGIN TRY
35     BEGIN TRANSACTION;
36     -- Archivar registros mayores a 180 días
37     INSERT INTO dbo.TramiteLogTemporalHistorico (IdLog, Descripcion, FechaRegistro)
38     SELECT IdLog, Descripcion, FechaRegistro
39     FROM dbo.TramiteLogTemporal
40     WHERE FechaRegistro < DATETIMEADD(DAY, -180, GETDATE());
41
42     SET @Filas = @@ROWCOUNT;
43
44     -- Eliminar registros mayores a 180 días
45     DELETE FROM dbo.TramiteLogTemporal
46     WHERE FechaRegistro < DATETIMEADD(DAY, -180, GETDATE());
47
48     -- Registrar en auditoría
49     INSERT INTO dbo.AuditoriaJobs (
50         NombreJob, Estado, Mensaje, RegistrosProcesados
51     ) VALUES (
52         N'JOB_Depurar_Tramite_Log_Temporal', N'CORRECTO',
53         N'Registros archivados y eliminados correctamente.', @Filas
54     );
55
56     COMMIT TRANSACTION;
57 END TRY
58 BEGIN CATCH
59     IF @@ROWCOUNT > 0
60     ROLLBACK TRANSACTION;
61
62     DECLARE @Error NVARCHAR(4000) = ERROR_MESSAGE();
63     INSERT INTO dbo.AuditoriaJobs (NombreJob, Estado, Mensaje, RegistrosProcesados)
64     VALUES (N'JOB_Depurar_Tramite_Log_Temporal', N'ERROR', @Error, 0);
65
66 END CATCH;
67
68 -- Fin del script
69
70 -- Mensajes
71
72 Commands completed successfully.
73 The job 'JOB_Depurar_Tramite_Log_Temporal' was successfully added.
74 The job step 'Archivar y eliminar registros antiguos' was successfully added.
75 The schedule 'SCH_Mensual_Depuracion_Log_Temporal' was successfully added.
76 The job 'JOB_Depurar_Tramite_Log_Temporal' was successfully attached to the schedule 'SCH_Mensual_Depuracion_Log_Temporal'.
77 The job 'JOB_Depurar_Tramite_Log_Temporal' was successfully added to the target server '(LOCAL)'.
78 Completion time: 2025-05-30T23:28:41.8732567-05:00
79
80 Query executed successfully. (LOCAL) (15.0 RTM) sa (59) GradosYTítulos 00:00:00 0 rows
```

Justificación técnica de la solución aplicada

La solución primero copia los registros antiguos a una tabla histórica y luego los elimina de la tabla temporal. Esto permite conservar información importante antes de realizar la depuración.

El uso de transacciones garantiza que ambas operaciones se realicen correctamente. Si ocurre un error, se revierte todo el proceso.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplica una transacción para proteger la integridad de los datos. También se usa TRY . . . CATCH para capturar errores y registrar el fallo.

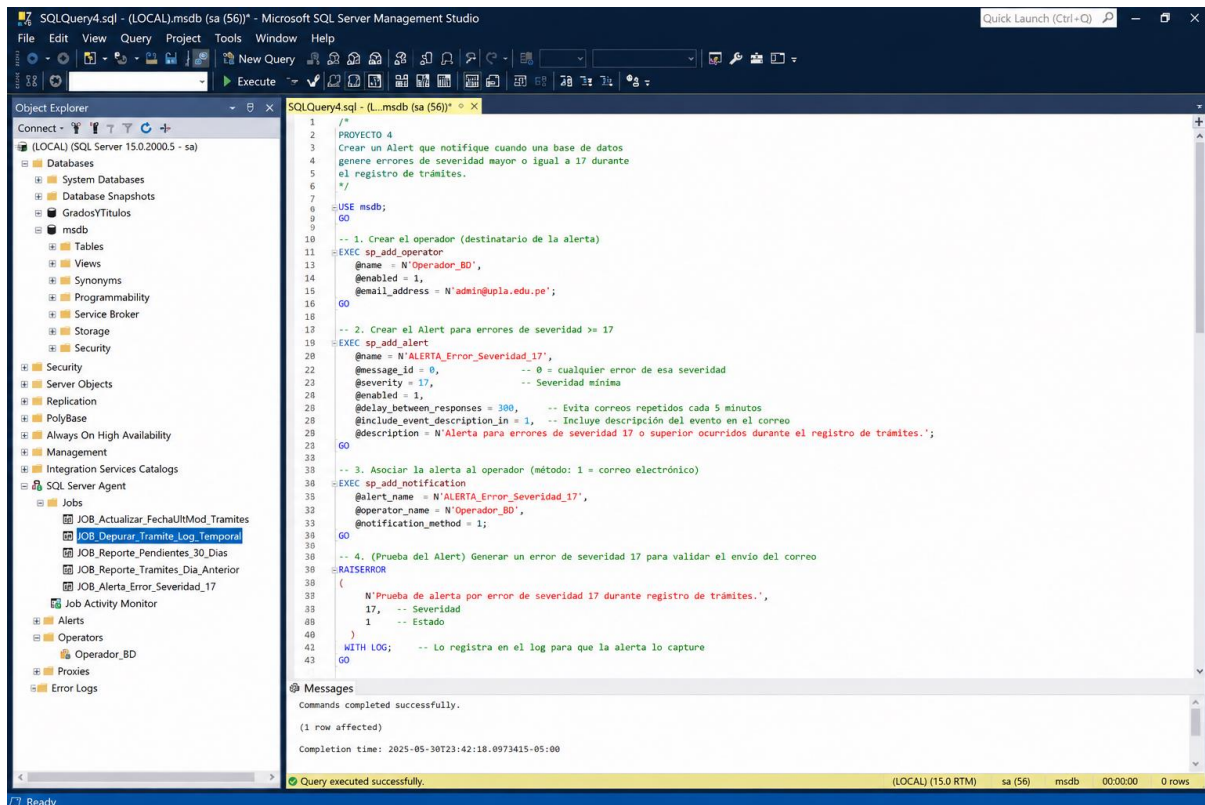
Además, la eliminación se programa mensualmente en un horario de baja carga, reduciendo el impacto en el rendimiento del sistema.

Proyecto 4

Enunciado del ejercicio

Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a **17** durante el registro de trámites.

Script de la solución en T-SQL



Justificación técnica de la solución aplicada

SQL Server Agent permite crear alertas asociadas a errores de severidad. Los errores de severidad 17 indican problemas importantes relacionados con recursos o condiciones internas del servidor.

La alerta se asocia a un operador para que el administrador reciba una notificación cuando ocurra el evento.

Explicación de las buenas prácticas utilizadas en el proyecto

Se configura un operador para centralizar las notificaciones. También se utiliza `delay_between_responses` para evitar que se envíen demasiadas alertas repetidas en poco tiempo.

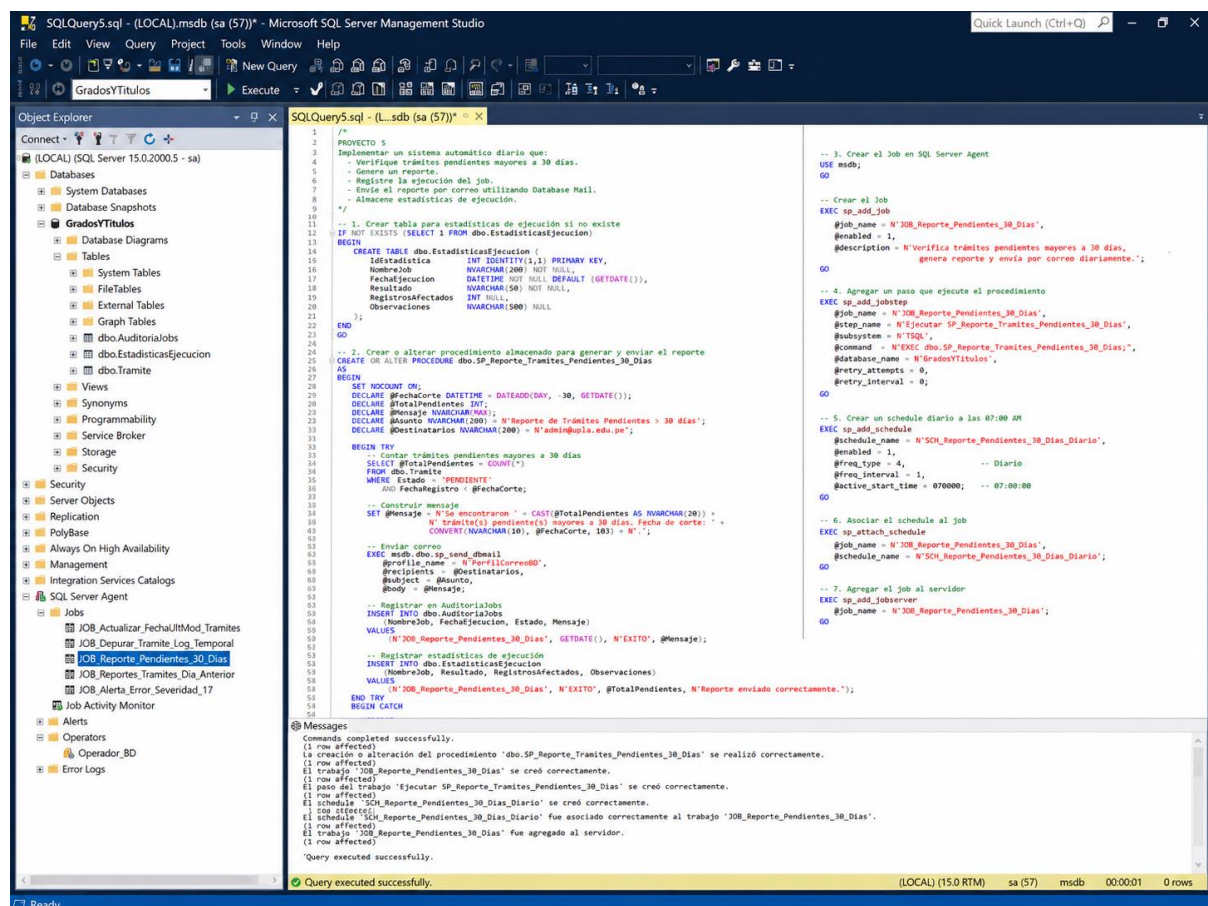
La prueba con `RAISERROR ... WITH LOG` permite validar que la alerta funciona correctamente.

Proyecto 5

Enunciado del ejercicio

Implementar un sistema automático que diariamente verifique trámites pendientes mayores a **30 días**, genere un reporte, registre la ejecución, envíe un correo mediante Database Mail y almacene estadísticas de ejecución.

Script de la solución en T-SQL



```
1  /*
2  PROYECTO 5
3  Implementar un sistema automatico diario que:
4  - Verifique trámites pendientes mayores a 30 días.
5  - Genere un reporte.
6  - Registre la ejecución del job.
7  - Envíe el reporte por correo utilizando Database Mail.
8  - Almacene estadísticas de ejecución.
9  */
10
11 -- 1. Crear tabla para estadísticas de ejecución si no existe
12 IF NOT EXISTS (SELECT 1 FROM dbo.EstadisticasEjecucion)
13 BEGIN
14     CREATE TABLE dbo.EstadisticasEjecucion (
15         IdEstadistica INT IDENTITY(1,1) PRIMARY KEY,
16         NombreJob NVARCHAR(200) NOT NULL,
17         FechaEjecucion DATETIME NOT NULL DEFAULT (GETDATE()),
18         Resultado NVARCHAR(50) NOT NULL,
19         RegistrosAfectados INT NULL,
20         Observaciones NVARCHAR(500) NULL
21     );
22 END
23 GO
24
25 -- 2. Crear o alterar procedimiento almacenado para generar y enviar el reporte
26 CREATE OR ALTER PROCEDURE dbo.SP_Reporte_Tramites_Pendientes_30_Dias
27 AS
28 BEGIN
29     SET NOCOUNT ON;
30     DECLARE @fechaCorte DATETIME = DATEADD(DAY, -30, GETDATE());
31     DECLARE @totalPendientes INT;
32     DECLARE @mensaje NVARCHAR(MAX);
33     DECLARE @asunto NVARCHAR(200) = N'Reporte de Trámites Pendientes > 30 días';
34     DECLARE @destinatarios NVARCHAR(200) = N'admin@pila.edu.pe';
35
36     BEGIN TRY
37         -- Contar trámites pendientes mayores a 30 días
38         SELECT @totalPendientes = COUNT(*)
39         FROM dbo.Tramite
40         WHERE Estado = 'PENDIENTE'
41         AND FechaRegistro < @fechaCorte;
42
43         -- Construir mensaje
44         SET @mensaje = N'Se encontraron ' + CAST(@totalPendientes AS NVARCHAR(20)) +
45             N'trámite(s) pendiente(s) mayores a 30 días. Fecha de corte: ' +
46             CONVERT(NVARCHAR(10), @fechaCorte, 103) + N'.';
47
48         -- Enviar correo
49         EXEC msdb.dbo.sp_send_dbmail
50             @profile_name = N'PerfilCorreoBD',
51             @recipients = @destinatarios,
52             @subject = @asunto,
53             @body = @mensaje;
54
55         -- Registrar en AuditorialJobs
56         INSERT INTO dbo.AuditorialJobs
57             (NombreJob, FechaEjecucion, Estado, Mensaje)
58         VALUES
59             (N'JOB_Reporte_Pendientes_30_Dias', GETDATE(), N'EXITO', @mensaje);
60
61         -- Registrar estadísticas de ejecución
62         INSERT INTO dbo.EstadisticasEjecucion
63             (NombreJob, Resultado, RegistrosAfectados, Observaciones)
64         VALUES
65             (N'JOB_Reporte_Pendientes_30_Dias', N'EXITO', @totalPendientes, N'Reporte enviado correctamente.');

Messages



Command completed successfully.



(1 row affected)



La creación o alteración del procedimiento 'dbo.SP_Reporte_Tramites_Pendientes_30_Dias' se realizó correctamente.



(1 row affected)



El trabajo 'JOB_Reporte_Pendientes_30_Dias' se creó correctamente.



(1 row affected)



El paso del trabajo 'Ejecutar SP_Reporte_Tramites_Pendientes_30_Dias' se creó correctamente.



(1 row affected)



El schedule 'SCH_Reporte_Pendientes_30_Dias_Diario' se creó correctamente.



(1 row affected)



El schedule 'SCH_Reporte_Pendientes_30_Dias_Diario' fue asociado correctamente al trabajo 'JOB_Reporte_Pendientes_30_Dias'.



(1 row affected)



El trabajo 'JOB_Reporte_Pendientes_30_Dias' fue agregado al servidor.



(1 row affected)



Query executed successfully.



Query executed successfully.


```

Justificación técnica de la solución aplicada

Se crea un procedimiento almacenado que identifica trámites pendientes con más de 30 días de antigüedad. Luego se envía un correo mediante `sp_send_dbmail`, se registra la ejecución en auditoría y se almacenan estadísticas del proceso.

El Job permite ejecutar este procedimiento automáticamente todos los días.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separa la lógica principal en un procedimiento almacenado, lo que facilita su mantenimiento y reutilización. También se usa TRY . . . CATCH para capturar errores.

El proceso registra estadísticas y auditoría, permitiendo analizar posteriormente si la ejecución fue correcta y cuántos trámites fueron detectados.

TEMA 2: Uso de Database Maintenance Plans

Nota: Se asume que la base de datos se llama GradosYTítulos. Las rutas como C:\Backups\ deben existir previamente en el servidor. Si usas otro nombre de base de datos o ruta, debes cambiarlo en el script.

Proyecto 1

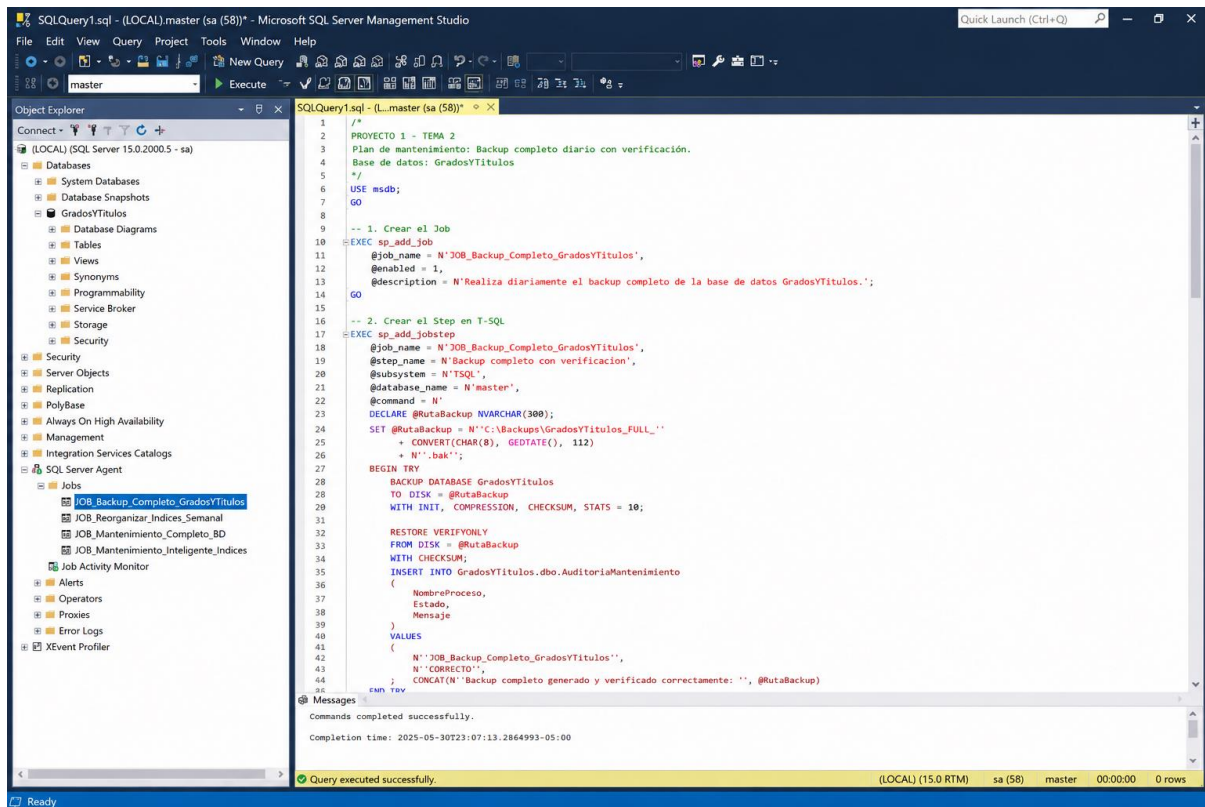
Enunciado del ejercicio

Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos GradosYTítulos.

El script debe incluir:

- BACKUP DATABASE
- Compresión
- Verificación
- Programación

Script de la solución en T-SQL



The screenshot shows the Microsoft SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with the 'Jobs' folder expanded under the 'SQL Server Agent' node. The 'JOB_Backup_Completo_GradosYTitulos' job is selected. The right pane shows the 'SQLQuery1.sql' script editor with the following T-SQL code:

```
1  /*
2  PROYECTO 1 - TEMA 2
3  Plan de mantenimiento: Backup completo diario con verificación.
4  Base de datos: GradosYTitulos
5  */
6  USE msdb;
7  GO
8
9  -- 1. Crear el Job
10 EXEC sp_add_job
11     @job_name = N'JOB_Backup_Completo_GradosYTitulos',
12     @enabled = 1,
13     @description = N'Realiza diariamente el backup completo de la base de datos GradosYTitulos.';
14 GO
15
16 -- 2. Crear el Step en T-SQL
17 EXEC sp_add_jobstep
18     @job_name = N'JOB_Backup_Completo_GradosYTitulos',
19     @step_name = N'Backup completo con verificación',
20     @subsystem = N'TSQL',
21     @database_name = N'master',
22     @command = N'
23 DECLARE @RutaBackup NVARCHAR(300);
24 SET @RutaBackup = N''C:\Backups\GradosYTitulos_FULL_' +
25     + CONVERT(CHAR(8), GETDATE(), 112)
26     + N''.bak'';
27 BEGIN TRY
28     BACKUP DATABASE GradosYTitulos
29     TO DISK = @RutaBackup
30     WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;
31
32     RESTORE VERIFYONLY
33     FROM DISK = @RutaBackup
34     WITH CHECKSUM;
35     INSERT INTO GradosYTitulos.dbo.AuditoriaMantenimiento
36     (
37         NombreProceso,
38         Estado,
39         Mensaje
40     )
41     VALUES
42     (
43         N'JOB_Backup_Completo_GradosYTitulos',
44         N'CORRECTO';
45         CONCAT(N'Backup completo generado y verificado correctamente: ', @RutaBackup)
46     )
47 END TRY
48 BEGIN CATCH
49     INSERT INTO GradosYTitulos.dbo.AuditoriaMantenimiento
50     (
51         NombreProceso,
52         Estado,
53         Mensaje
54     )
55     VALUES
56     (
57         N'JOB_Backup_Completo_GradosYTitulos',
58         N'ERROR';
59         ERROR_MESSAGE()
60     )
61 END CATCH
62 GO
```

The bottom status bar indicates the query was executed successfully on the 'master' database at 00:00:00.

Justificación técnica de la solución aplicada

Se utiliza un Job de SQL Server Agent para ejecutar diariamente un respaldo completo de la base de datos. El comando `BACKUP DATABASE` genera la copia de seguridad, mientras que `COMPRESSION` reduce el tamaño del archivo.

La opción `CHECKSUM` ayuda a validar la consistencia del backup, y `RESTORE VERIFYONLY` permite comprobar que el archivo generado puede ser leído correctamente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se programa el backup en horario de baja carga, en este caso a las 02:00 AM. También se usa compresión para ahorrar espacio en disco y verificación para reducir el riesgo de almacenar respaldos dañados.

Además, se registra el resultado del proceso en una tabla de auditoría, permitiendo verificar si el mantenimiento se ejecutó correctamente.

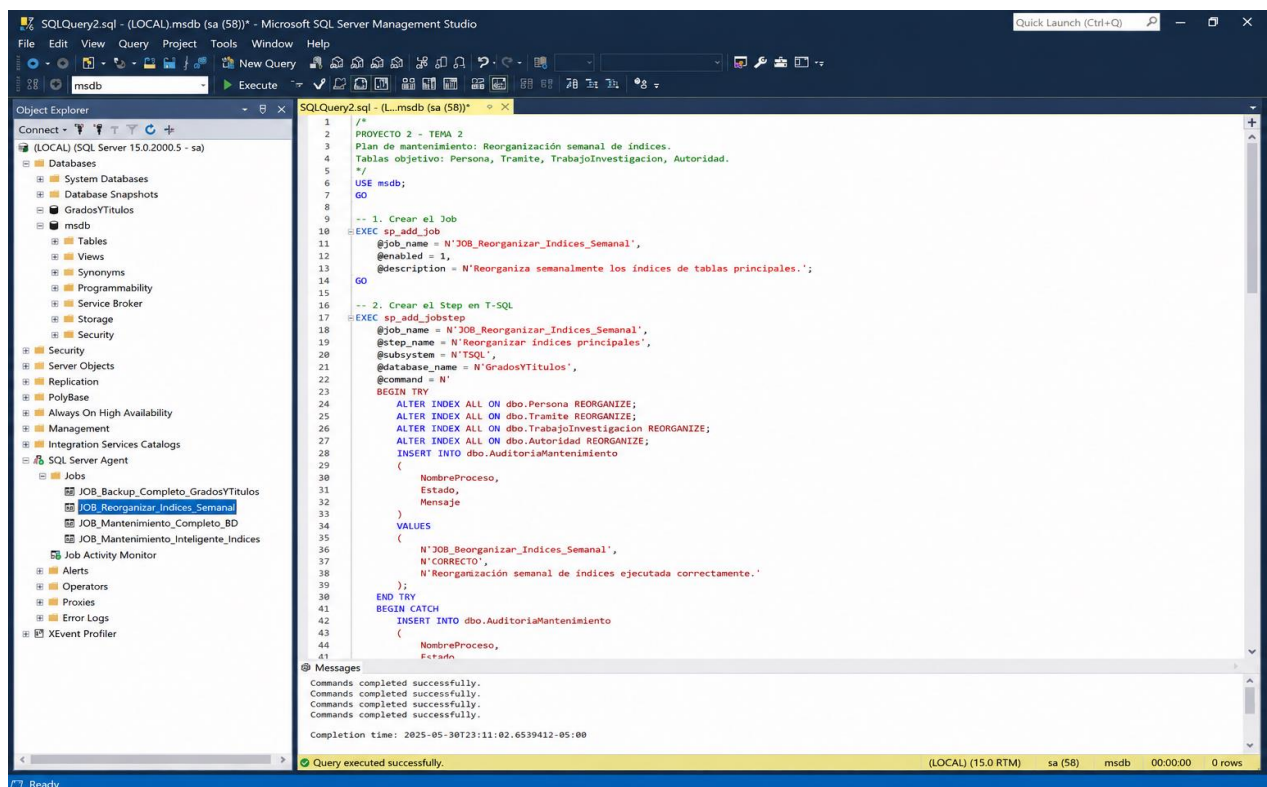
Proyecto 2

Enunciado del ejercicio

Crear un plan semanal para reorganizar índices de las tablas:

- Persona
- Tramite
- TrabajoInvestigacion
- Autoridad

Script de la solución en T-SQL



Justificación técnica de la solución aplicada

La reorganización de índices permite reducir la fragmentación moderada sin reconstruir completamente la estructura del índice. Esto mejora el rendimiento de las consultas y reduce el tiempo de lectura de datos.

Se aplica a tablas importantes del sistema académico, ya que probablemente son consultadas con frecuencia.

Explicación de las buenas prácticas utilizadas en el proyecto

Se ejecuta semanalmente para no sobrecargar el servidor todos los días. Además, se programa en horario de baja actividad y se registra la ejecución en auditoría.

El uso de `ALTER INDEX ALL ... REORGANIZE` permite aplicar mantenimiento a todos los índices de cada tabla indicada.

Proyecto 3

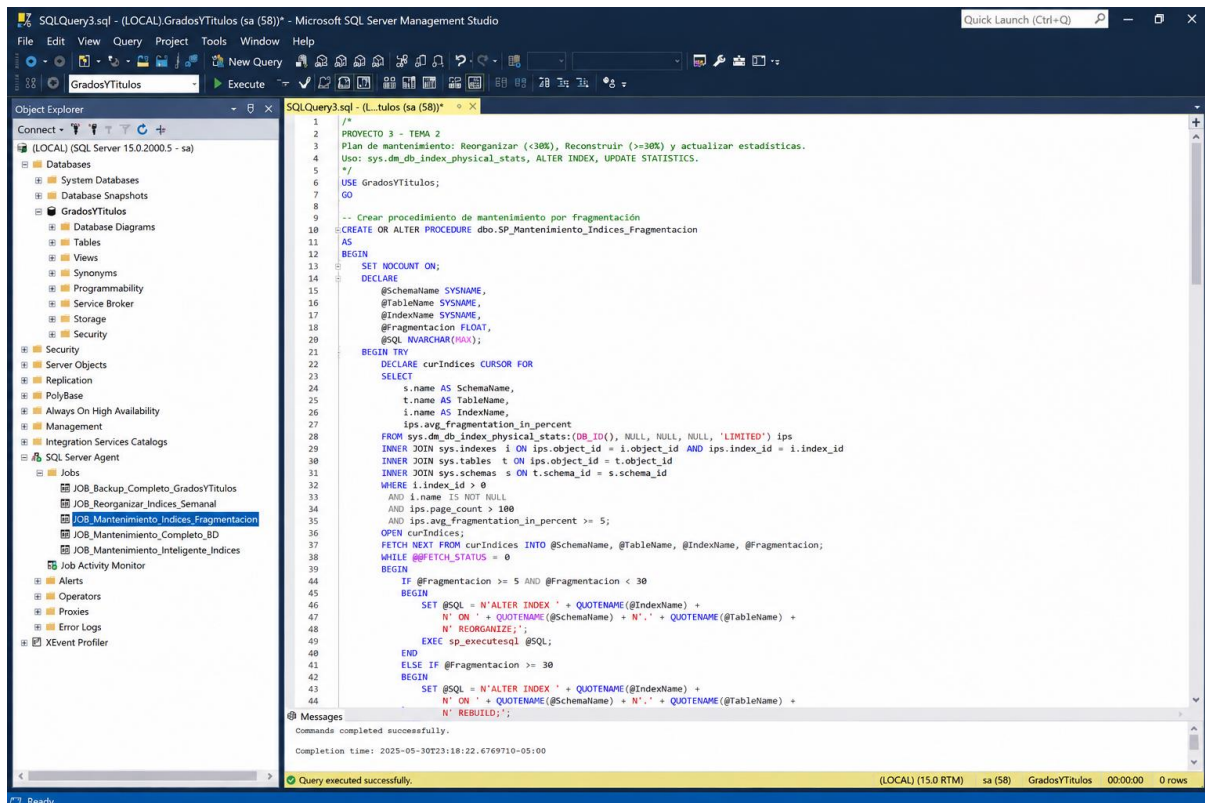
Enunciado del ejercicio

Diseñar un plan que reorganice índices menores al 30%, reconstruya índices mayores al 30% y actualice estadísticas.

El script debe utilizar:

- `sys.dm_db_index_physical_stats`
- `ALTER INDEX`
- `UPDATE STATISTICS`

Script de la solución en T-SQL



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the database structure for 'GradosYTítulos'. The main window shows a T-SQL script titled 'SQLQuery3.sql - (LOCAL)GradosYTítulos (sa (58))'. The script is a stored procedure named 'dbo.SP_Mantenimiento_Indices_Fragmentacion' that iterates through indices, checks their fragmentation level, and performs maintenance (REORGANIZE or REBUILD) based on the fragmentation percentage. The script also updates statistics. The status bar at the bottom indicates the query was executed successfully.

```
1 /*
2 PROYECTO 3 - TEMA 2
3 Plan de mantenimiento: Reorganizar (<30%), Reconstruir (>=30%) y actualizar estadísticas.
4 Usos: sys.dm_db_index_physical_stats, ALTER INDEX, UPDATE STATISTICS.
5 */
6 USE GradosYTítulos;
7 GO
8
9 -- Crear procedimiento de mantenimiento por fragmentación
10 CREATE OR ALTER PROCEDURE dbo.SP_Mantenimiento_Indices_Fragmentacion
11 AS
12 BEGIN
13     SET NOCOUNT ON;
14     DECLARE
15         @SchemaName SYSNAME,
16         @TableName SYSNAME,
17         @IndexName SYSNAME,
18         @Fragmentacion FLOAT,
19         @SQL NVARCHAR(MAX);
20
21     BEGIN TRY
22         DECLARE curIndices CURSOR FOR
23         SELECT
24             s.name AS SchemaName,
25             t.name AS TableName,
26             i.name AS IndexName,
27             ips.avg_fragmentation_in_percent
28         FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
29         INNER JOIN sys.indexes i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
30         INNER JOIN sys.tables t ON ips.object_id = t.object_id
31         INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
32         WHERE i.index_id > 0
33             AND i.name IS NOT NULL
34             AND ips.page_count > 100
35             AND ips.avg_fragmentation_in_percent >= 5;
36         OPEN curIndices;
37         FETCH NEXT FROM curIndices INTO @SchemaName, @TableName, @IndexName, @Fragmentacion;
38         WHILE @@FETCH_STATUS = 0
39         BEGIN
40             IF @Fragmentacion >= 5 AND @Fragmentacion < 30
41             BEGIN
42                 SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName) +
43                     N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) +
44                     N' REORGANIZE;';
45                 EXEC sp_executesql @SQL;
46             END
47             ELSE IF @Fragmentacion >= 30
48             BEGIN
49                 SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName) +
50                     N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) +
51                     N' REBUILD;';
52                 EXEC sp_executesql @SQL;
53             END
54             FETCH NEXT FROM curIndices INTO @SchemaName, @TableName, @IndexName, @Fragmentacion;
55         END
56     END TRY
57     BEGIN CATCH
58         PRINT 'Error en el procedimiento de mantenimiento de índices.';
59     END CATCH
60 END
```

Justificación técnica de la solución aplicada

Se utiliza la DMV `sys.dm_db_index_physical_stats` para obtener el porcentaje de fragmentación de los índices. Según el nivel encontrado, el procedimiento decide si debe reorganizar o reconstruir el índice.

También se actualizan estadísticas para que SQL Server pueda generar mejores planes de ejecución en futuras consultas.

Explicación de las buenas prácticas utilizadas en el proyecto

Se evita realizar mantenimiento sobre índices pequeños mediante la condición `page_count > 100`. También se usa `QUOTENAME` para construir SQL dinámico de manera segura.

El mantenimiento se aplica solo cuando la fragmentación supera el 5%, evitando operaciones innecesarias.

Proyecto 4

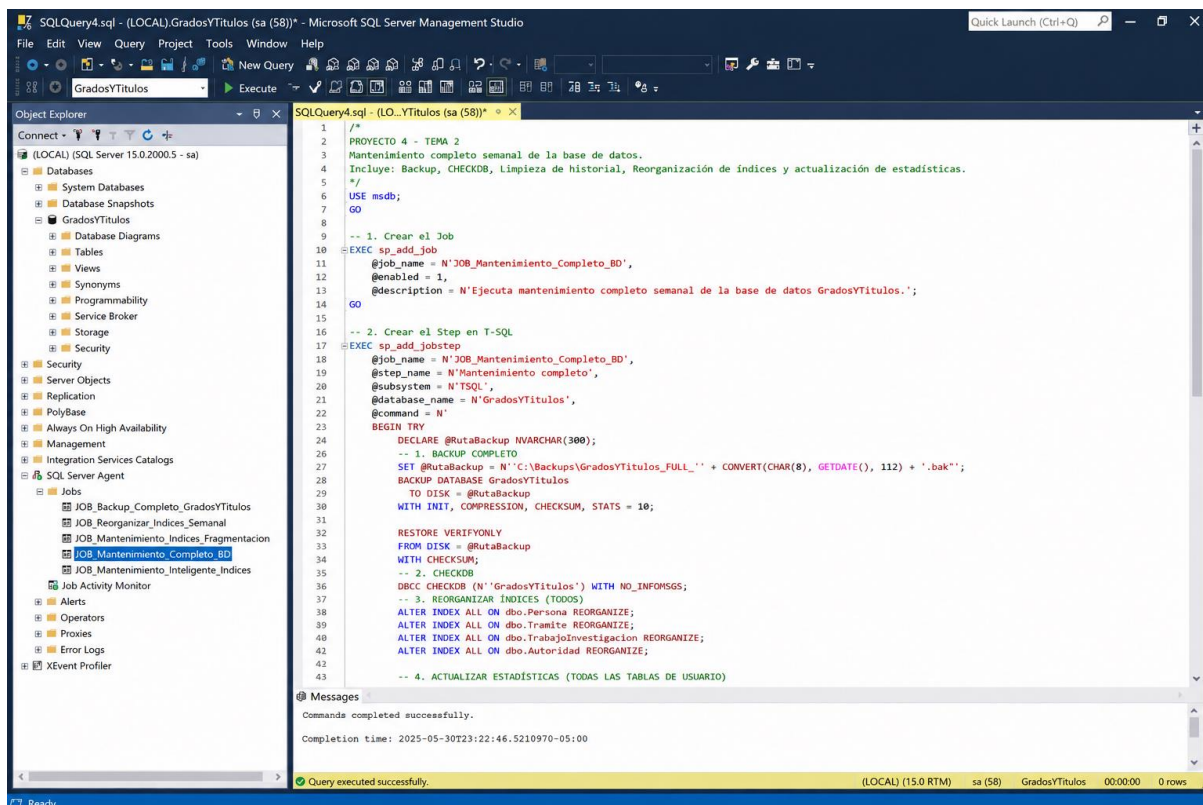
Enunciado del ejercicio

Automatizar el mantenimiento completo de la base de datos incluyendo:

- Backup
- CHECKDB
- Limpieza del historial
- Reorganización de índices
- Actualización de estadísticas

El script debe utilizar Maintenance Plans y T-SQL.

Script de la solución en T-SQL



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the 'GradosYTítulos' database selected under the 'Databases' folder. The main window shows a T-SQL script titled 'SQLQuery4.sql - (LOCAL)GradosYTítulos (sa (58))'. The script is designed to automate database maintenance tasks. It starts with a comment block describing the project and the tasks to be performed: Backup, CHECKDB, cleaning the history, reorganizing indices, and updating statistics. The script then proceeds to create a job named 'JOB_Mantenimiento_Completo_BD' and a job step named 'Mantenimiento completo'. The job step's command is a T-SQL script that performs the following tasks: 1. Backup: A full backup of the 'GradosYTítulos' database to a specified path, with compression and a backup name that includes the date. 2. CHECKDB: A database consistency check with no informational messages. 3. Reorganizing indices: Reorganizes all indices in the 'dbo' schema. 4. Updating statistics: Updates statistics for all user tables. The script concludes with a message box indicating that the commands were completed successfully.

```
1  /*
2  PROYECTO 4 - TEMA 2
3  Mantenimiento completo semanal de la base de datos.
4  Incluye: Backup, CHECKDB, Limpieza de historial, Reorganización de índices y actualización de estadísticas.
5  */
6  USE msdb;
7  GO
8
9  -- 1. Crear el Job
10 @EXEC sp_add_job
11 @job_name = N'JOB_Mantenimiento_Completo_BD',
12 @enabled = 1,
13 @description = N'Ejecuta mantenimiento completo semanal de la base de datos GradosYTítulos.';
14 GO
15
16 -- 2. Crear el Step en T-SQL
17 @EXEC sp_add_jobstep
18 @job_name = N'JOB_Mantenimiento_Completo_BD',
19 @step_name = N'Mantenimiento completo',
20 @subsystem = N'TSQL',
21 @database_name = N'GradosYTítulos',
22 @command = N'
23 BEGIN TRY
24 DECLARE @RutaBackup NVARCHAR(300);
25 -- 1. BACKUP COMPLETO
26 SET @RutaBackup = N'C:\Backups\GradosYTítulos_FULL_' + CONVERT(CHAR(8), GETDATE(), 112) + '.bak';
27 BACKUP DATABASE GradosYTítulos
28 TO DISK = @RutaBackup
29 WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;
30
31 RESTORE VERIFYONLY
32 FROM DISK = @RutaBackup
33 WITH CHECKSUM;
34
35 -- 2. CHECKDB
36 DBCC CHECKDB (N'GradosYTítulos') WITH NO_INFOMSGS;
37
38 -- 3. REORGANIZAR INDICES (TODOS)
39 ALTER INDEX ALL ON dbo.Persona REORGANIZE;
40 ALTER INDEX ALL ON dbo.Trámite REORGANIZE;
41 ALTER INDEX ALL ON dbo.TrabajoInvestigación REORGANIZE;
42 ALTER INDEX ALL ON dbo.Autoridad REORGANIZE;
43
44 -- 4. ACTUALIZAR ESTADÍSTICAS (TODAS LAS TABLAS DE USUARIO)
45
```

Justificación técnica de la solución aplicada

El procedimiento integra las principales tareas de mantenimiento preventivo de una base de datos. Primero genera un backup completo, luego valida el archivo, ejecuta CHECKDB, realiza mantenimiento de índices y limpia historiales antiguos.

Al ejecutarse mediante SQL Server Agent, funciona como un plan de mantenimiento automatizado.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centraliza el mantenimiento en un procedimiento almacenado para facilitar su administración. Además, se verifica el backup, se revisa la integridad de la base de datos y se eliminan historiales antiguos para evitar crecimiento innecesario.

También se registra el resultado en auditoría para tener trazabilidad del mantenimiento.

Proyecto 5

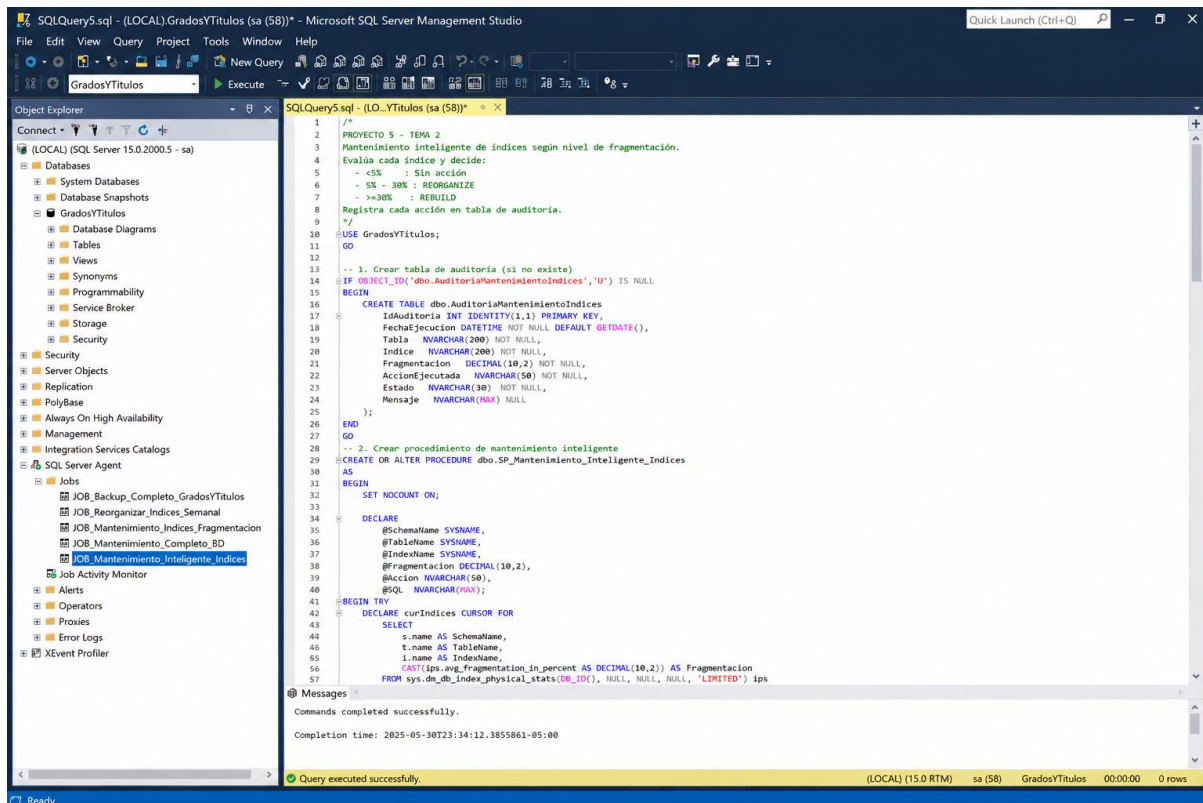
Enunciado del ejercicio

Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.

El script debe utilizar:

- Cursores
- DMV
- ALTER INDEX
- UPDATE STATISTICS
- Tablas de auditoría

Script de la solución en T-SQL



```
1  /*
2  PROYECTO 5 - TEMA 2
3  Mantenimiento inteligente de índices según nivel de fragmentación.
4  Evalúa cada índice y decide:
5  -- <3% : Sin acción
6  -- SS < 30% : REORGANIZE
7  -- >=30% : REBUILD
8  Registra cada acción en tabla de auditoría.
9  */
10
11 USE GradosYTítulos;
12 GO
13
14 -- 1. Crear tabla de auditoría (si no existe)
15 IF OBJECT_ID('dbo.AuditoriaMantenimientoIndices','U') IS NULL
16 BEGIN
17     CREATE TABLE dbo.AuditoriaMantenimientoIndices
18     (
19         IDAuditoria INT IDENTITY(1,1) PRIMARY KEY,
20         FechaEjecucion DATETIME NOT NULL DEFAULT GETDATE(),
21         Tabla NVARCHAR(200) NOT NULL,
22         Indice NVARCHAR(200) NOT NULL,
23         Fragmentacion DECIMAL(10,2) NOT NULL,
24         AccionEjecutada NVARCHAR(50) NOT NULL,
25         Estado NVARCHAR(30) NOT NULL,
26         Mensaje NVARCHAR(MAX) NULL
27     );
28 END
29 GO
30
31 -- 2. Crear procedimiento de mantenimiento inteligente
32 CREATE OR ALTER PROCEDURE dbo.SP_Mantenimiento_Inteligente_Indices
33 AS
34 BEGIN
35     SET NOCOUNT ON;
36
37     DECLARE
38         @SchemaName SYSNAME,
39         @TableName SYSNAME,
40         @IndexName SYSNAME,
41         @Fragmentacion DECIMAL(10,2),
42         @Accion NVARCHAR(50),
43         @SQL NVARCHAR(MAX);
44
45     BEGIN TRY
46         DECLARE curIndices CURSOR FOR
47         SELECT
48             s.name AS SchemaName,
49             t.name AS TableName,
50             i.name AS IndexName,
51             CAST((ips.avg_fragmentation_in_percent AS DECIMAL(10,2)) AS Fragmentacion
52             FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
53     )
```

Justificación técnica de la solución aplicada

El plan inteligente consulta la fragmentación real de los índices mediante una DMV. Según el porcentaje obtenido, decide automáticamente si debe reorganizar, reconstruir o no realizar ninguna acción.

Esto permite que el mantenimiento sea más eficiente, porque no aplica la misma operación a todos los índices.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizan umbrales técnicos para evitar operaciones innecesarias. Los índices con fragmentación baja no se modifican, los índices con fragmentación moderada se reorganizan y los índices con fragmentación alta se reconstruyen.

También se registra cada índice evaluado en una tabla de auditoría, permitiendo revisar qué acciones fueron ejecutadas durante el mantenimiento.

TEMA 3: Automatización con T-SQL y PowerShell

Nota: Se asume que la base de datos se llama GradosYTítulos. Las rutas como C:\Backups\, C:\Reportes\ y C:\Respaldos\ deben existir previamente. También se asume que PowerShell tiene disponible el módulo SqlServer para usar Invoke-Sqlcmd.

Proyecto 1

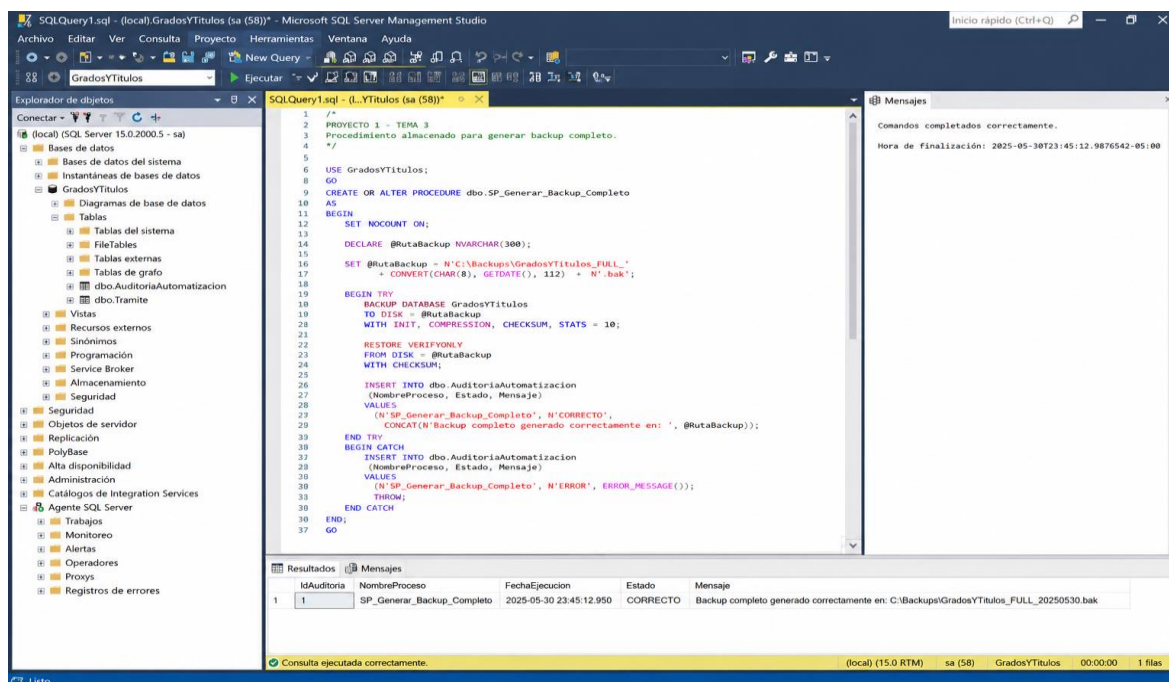
Enunciado del ejercicio

Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell.

El script debe desarrollar:

- Stored Procedure.
- BACKUP DATABASE.
- PowerShell Invoke-Sqlcmd.

Script de la solución en T-SQL



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'GradosYTítulos' database selected. The central pane shows a T-SQL script named 'SQLQuery1.sql' with the following content:

```
1 /*
2 PROYECTO 1 - TEMA 3
3 Procedimiento almacenado para generar backup completo.
4 */
5
6 USE GradosYTítulos;
7 GO
8
9 CREATE OR ALTER PROCEDURE dbo.SP_Generar_Backup_Completo
10 AS
11 BEGIN
12     SET NOCOUNT ON;
13
14     DECLARE @RutaBackup NVARCHAR(300);
15
16     SET @RutaBackup = N'C:\Backups\GradosYTítulos_FULL_'
17     + CONVERT(CHAR(8), GETDATE(), 112) + N'.bak';
18
19     BEGIN TRY
20         BACKUP DATABASE GradosYTítulos
21         TO DISK = @RutaBackup
22         WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;
23
24         RESTORE VERIFYONLY
25         FROM DISK = @RutaBackup
26         WITH CHECKSUM;
27
28         INSERT INTO dbo.AuditoriaAutomatizacion
29         (NombreProceso, Estado, Mensaje)
30         VALUES
31         (N'SP_Generar_Backup_Completo', N'CORRECTO',
32          CONCAT(N'Backup completo generado correctamente en: ', @RutaBackup));
33     END TRY
34     BEGIN CATCH
35         INSERT INTO dbo.AuditoriaAutomatizacion
36         (NombreProceso, Estado, Mensaje)
37         VALUES
38         (N'SP_Generar_Backup_Completo', N'ERROR', ERROR_MESSAGE());
39     END CATCH
40 END;
41 GO
```

The right pane shows the 'Mensajes' (Messages) window with the following text:

Comandos completados correctamente.
Hora de finalización: 2025-05-30T23:45:12.9876542-05:00

The bottom pane shows the 'Resultados' (Results) window with the following data:

IdAuditoria	NombreProceso	FechaEjecucion	Estado	Mensaje
1	SP_Generar_Backup_Completo	2025-05-30 23:45:12.950	CORRECTO	Backup completo generado correctamente en: C:\Backups\GradosYTítulos_FULL_20250530.bak

The status bar at the bottom indicates 'Consulta ejecutada correctamente.' (Query executed successfully.)

Justificación técnica de la solución aplicada

Se crea un procedimiento almacenado para centralizar la generación del respaldo completo de la base de datos. Luego, PowerShell ejecuta dicho procedimiento mediante `Invoke-Sqlcmd`, permitiendo automatizar la tarea desde el sistema operativo.

El uso de `BACKUP DATABASE` permite proteger la información, mientras que `RESTORE VERIFYONLY` valida que el archivo de respaldo pueda ser leído correctamente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza compresión para reducir el tamaño del backup y `CHECKSUM` para mejorar la validación del archivo. Además, se registra el resultado en una tabla de auditoría.

Separar la lógica SQL en un procedimiento almacenado facilita el mantenimiento y permite que PowerShell solo se encargue de ejecutar el proceso.

Proyecto 2

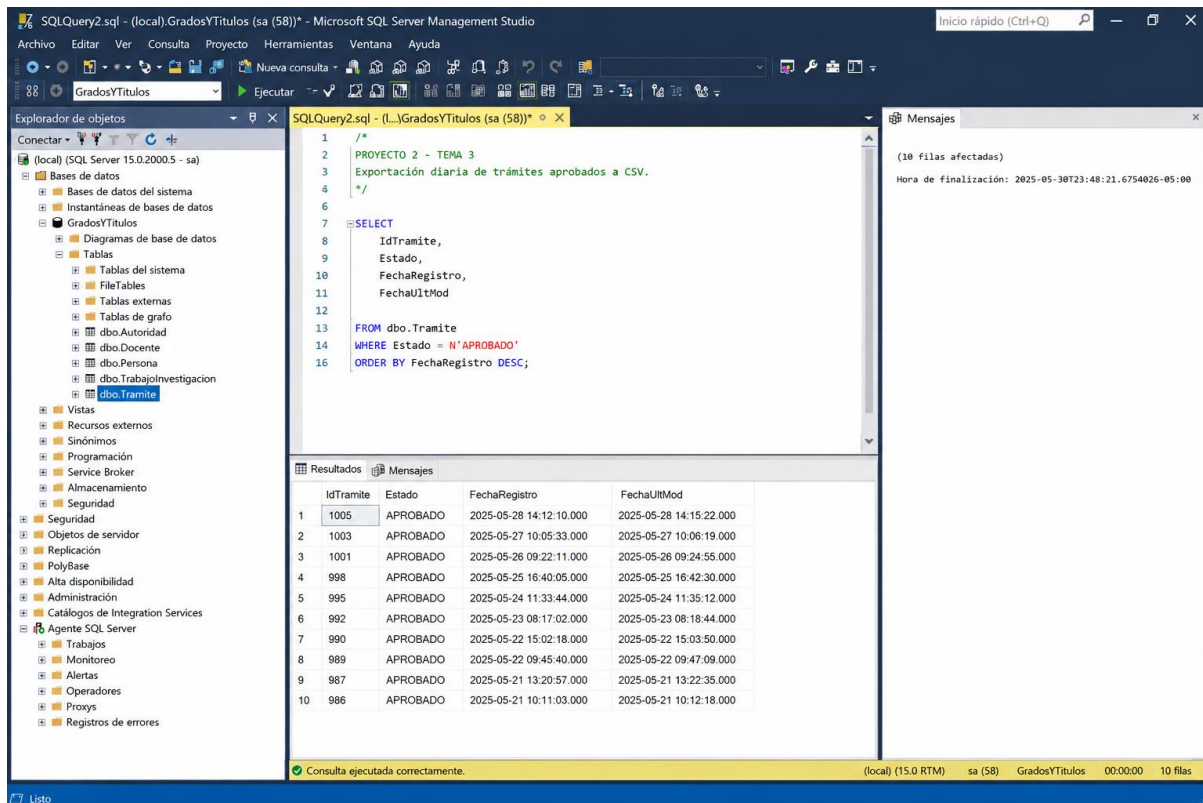
Enunciado del ejercicio

Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.

El script debe utilizar:

- `SELECT`.
- `Invoke-Sqlcmd`.
- `Export-Csv`.

Script de la solución en T-SQL



The screenshot displays the Microsoft SQL Server Management Studio interface. The central pane shows a T-SQL query script for 'SQLQuery2.sql'. The query is a SELECT statement that filters for 'APROBADO' records from the 'dbo.Tramite' table, ordered by 'FechaRegistro' in descending order. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'GradosYTítulos' database selected. The bottom pane shows the 'Resultados' (Results) tab with a table of 10 rows. The right pane shows the 'Mensajes' (Messages) tab with a confirmation message.

```
1 /*
2 PROYECTO 2 - TEMA 3
3 Exportación diaria de trámites aprobados a CSV.
4 */
5
6
7 SELECT
8     IdTramite,
9     Estado,
10    FechaRegistro,
11    FechaUltMod
12
13 FROM dbo.Tramite
14 WHERE Estado = N'APROBADO'
15 ORDER BY FechaRegistro DESC;
```

	IdTramite	Estado	FechaRegistro	FechaUltMod
1	1005	APROBADO	2025-05-28 14:12:10.000	2025-05-28 14:15:22.000
2	1003	APROBADO	2025-05-27 10:05:33.000	2025-05-27 10:06:19.000
3	1001	APROBADO	2025-05-26 09:22:11.000	2025-05-26 09:24:55.000
4	998	APROBADO	2025-05-25 16:40:05.000	2025-05-25 16:42:30.000
5	995	APROBADO	2025-05-24 11:33:44.000	2025-05-24 11:35:12.000
6	992	APROBADO	2025-05-23 08:17:02.000	2025-05-23 08:18:44.000
7	990	APROBADO	2025-05-22 15:02:18.000	2025-05-22 15:03:50.000
8	989	APROBADO	2025-05-22 09:45:40.000	2025-05-22 09:47:09.000
9	987	APROBADO	2025-05-21 13:20:57.000	2025-05-21 13:22:35.000
10	986	APROBADO	2025-05-21 10:11:03.000	2025-05-21 10:12:18.000

Mensajes (10 filas afectadas)
Hora de finalización: 2025-05-30T23:48:21.6754026+05:00

Consulta ejecutada correctamente. (local) (15.0 RTM) sa (58) GradosYTítulos 00:00:00 10 filas

Justificación técnica de la solución aplicada

La consulta T-SQL obtiene únicamente los trámites aprobados de la base de datos. Luego, PowerShell ejecuta la consulta con `Invoke-Sqlcmd` y exporta los resultados a un archivo CSV mediante `Export-Csv`.

El formato CSV es adecuado porque puede abrirse fácilmente en Excel u otras herramientas de análisis.

Explicación de las buenas prácticas utilizadas en el proyecto

Se filtran solo los registros necesarios mediante `WHERE Estado = 'APROBADO'`, evitando exportar información innecesaria. Además, el archivo generado incluye la fecha en el nombre, lo que permite organizar los reportes diarios.

Se usa codificación UTF8 para evitar problemas con caracteres especiales.

Proyecto 3

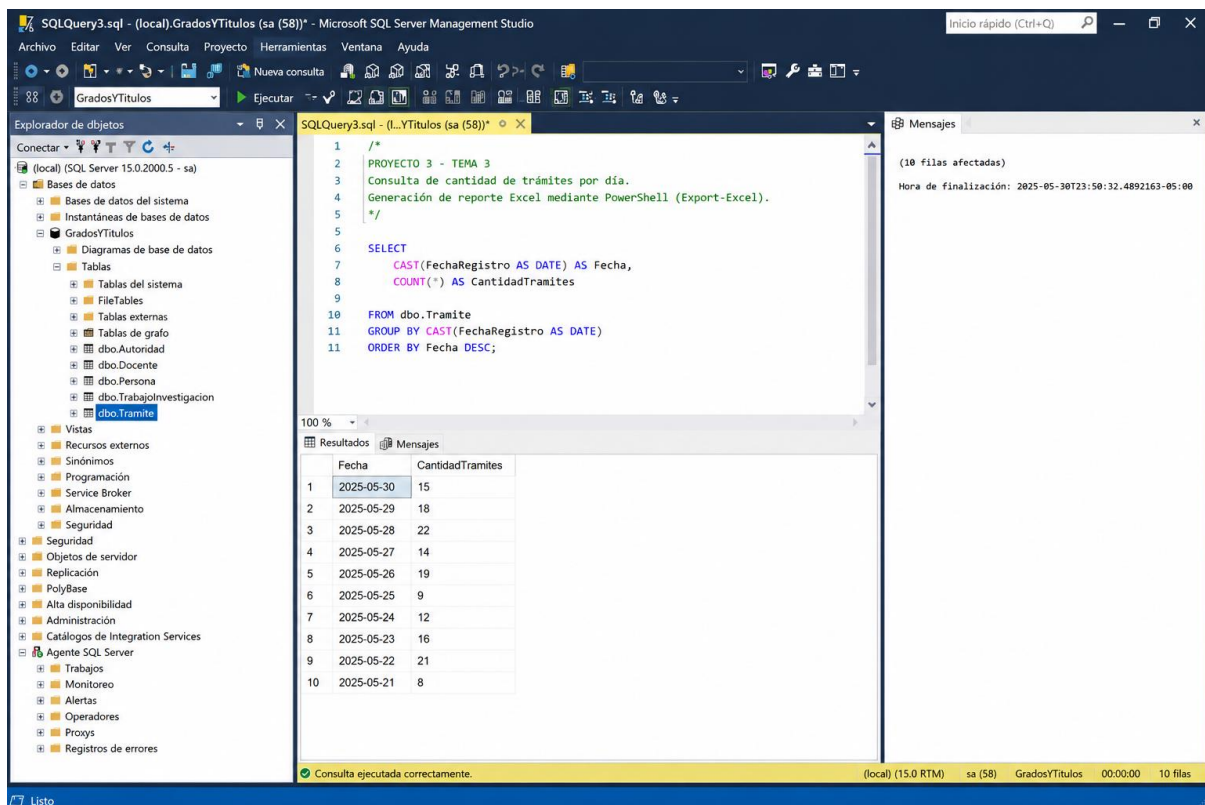
Enunciado del ejercicio

Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel y almacene una copia en una carpeta de respaldo.

El script debe utilizar:

- T-SQL.
- PowerShell.
- Export -Excel del módulo ImportExcel.

Script de la solución en T-SQL



The screenshot shows the Microsoft SQL Server Management Studio interface. The query editor displays a T-SQL script for Project 3, which includes a comment about the purpose of the query and a SELECT statement to retrieve the number of processes per date from the 'Tramite' table. The results pane shows the output of the query, displaying 10 rows of data with columns 'Fecha' and 'CantidadTramites'. The status bar at the bottom indicates that the query was executed successfully.

```
1  /*
2  PROYECTO 3 - TEMA 3
3  Consulta de cantidad de trámites por día.
4  Generación de reporte Excel mediante PowerShell (Export-Excel).
5  */
6
7  SELECT
8      CAST(FechaRegistro AS DATE) AS Fecha,
9      COUNT(*) AS CantidadTramites
10 FROM dbo.Tramite
11 GROUP BY CAST(FechaRegistro AS DATE)
12 ORDER BY Fecha DESC;
```

	Fecha	CantidadTramites
1	2025-05-30	15
2	2025-05-29	18
3	2025-05-28	22
4	2025-05-27	14
5	2025-05-26	19
6	2025-05-25	9
7	2025-05-24	12
8	2025-05-23	16
9	2025-05-22	21
10	2025-05-21	8

Consulta ejecutada correctamente. (local) (15.0 RTM) sa (58) GradosYTítulos 00:00:00 10 filas

Justificación técnica de la solución aplicada

La consulta agrupa los trámites por fecha y calcula la cantidad registrada en cada día. PowerShell ejecuta la consulta, genera un archivo Excel con Export - Excel y luego almacena una copia en una carpeta de respaldo.

Esto permite contar con un reporte diario en un formato más presentable que CSV.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separa la generación del reporte de la copia de respaldo. También se utiliza un nombre de archivo con fecha para evitar sobrescritura accidental.

El uso de Export - Excel permite generar reportes ordenados, legibles y útiles para revisión administrativa.

Proyecto 4

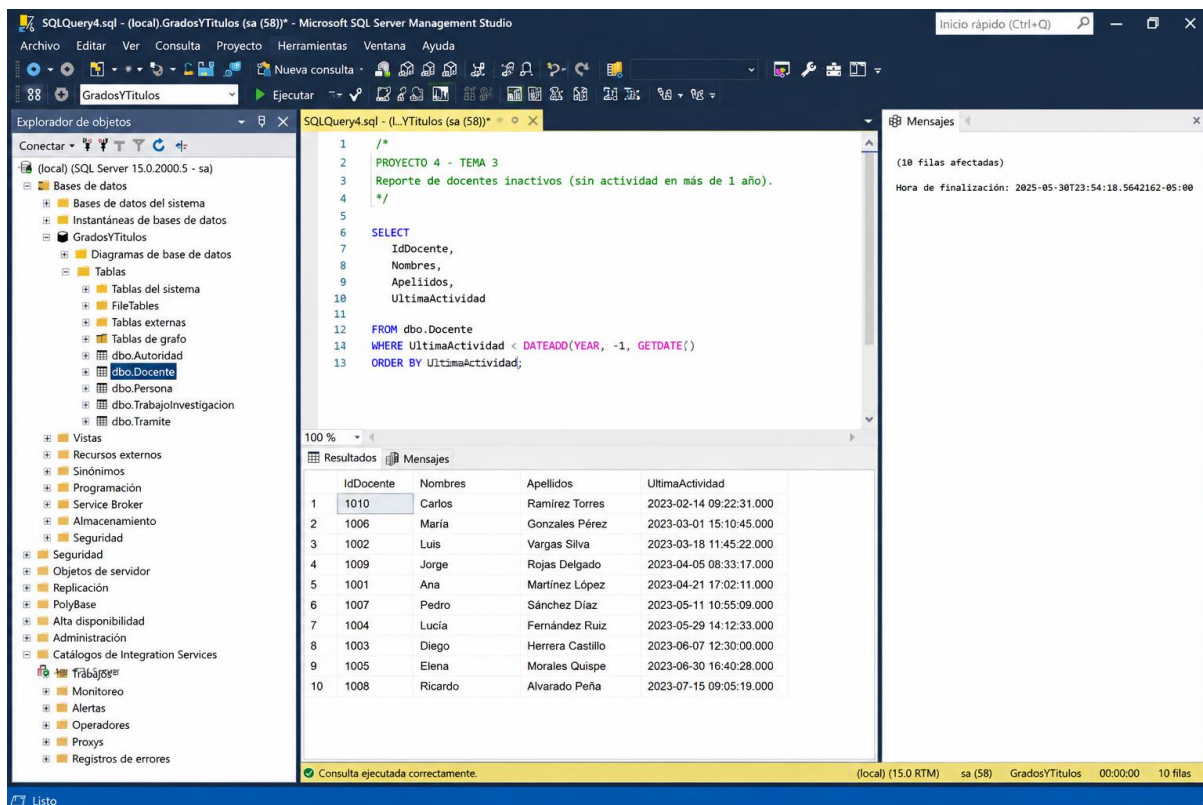
Enunciado del ejercicio

Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo con el reporte.

El script debe implementar:

- Consultas T-SQL.
- PowerShell.
- Exportación automática.

Script de la solución en T-SQL



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'GradosYTítulos' database selected. The central pane shows a T-SQL query in the 'SQLQuery4.sql' file. The query is a SELECT statement that filters for inactive teachers based on their last activity date. The right pane shows the 'Mensajes' (Messages) window, indicating that 10 rows were affected. Below the query, the 'Resultados' (Results) window displays a table with 10 rows of data.

```
1  /*
2  PROYECTO 4 - TEMA 3
3  Reporte de docentes inactivos (sin actividad en más de 1 año).
4  */
5
6  SELECT
7      IdDocente,
8      Nombres,
9      Apellidos,
10     UltimaActividad
11
12 FROM dbo.Docente
13 WHERE UltimaActividad < DATEADD(YEAR, -1, GETDATE())
14 ORDER BY UltimaActividad;
```

	IdDocente	Nombres	Apellidos	UltimaActividad
1	1010	Carlos	Ramírez Torres	2023-02-14 09:22:31.000
2	1006	María	Gonzales Pérez	2023-03-01 15:10:45.000
3	1002	Luis	Vargas Silva	2023-03-18 11:45:22.000
4	1009	Jorge	Rojas Delgado	2023-04-05 08:33:17.000
5	1001	Ana	Martínez López	2023-04-21 17:02:11.000
6	1007	Pedro	Sánchez Díaz	2023-05-11 10:55:09.000
7	1004	Lucía	Fernández Ruiz	2023-05-29 14:12:33.000
8	1003	Diego	Herrera Castillo	2023-06-07 12:30:00.000
9	1005	Elena	Morales Quispe	2023-06-30 16:40:28.000
10	1008	Ricardo	Alvarado Peña	2023-07-15 09:05:19.000

Justificación técnica de la solución aplicada

La consulta identifica a los docentes cuya última actividad fue registrada hace más de un año. Luego, PowerShell genera automáticamente un archivo CSV con la información encontrada.

Este proceso permite mantener actualizado el control administrativo de docentes activos e inactivos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una condición basada en fechas mediante DATEADD, lo que permite calcular dinámicamente el periodo de inactividad. Además, se genera un archivo con fecha para mantener el historial de reportes.

El proceso puede programarse diariamente, semanalmente o mensualmente según la necesidad de la institución.

Proyecto 5

Enunciado del ejercicio

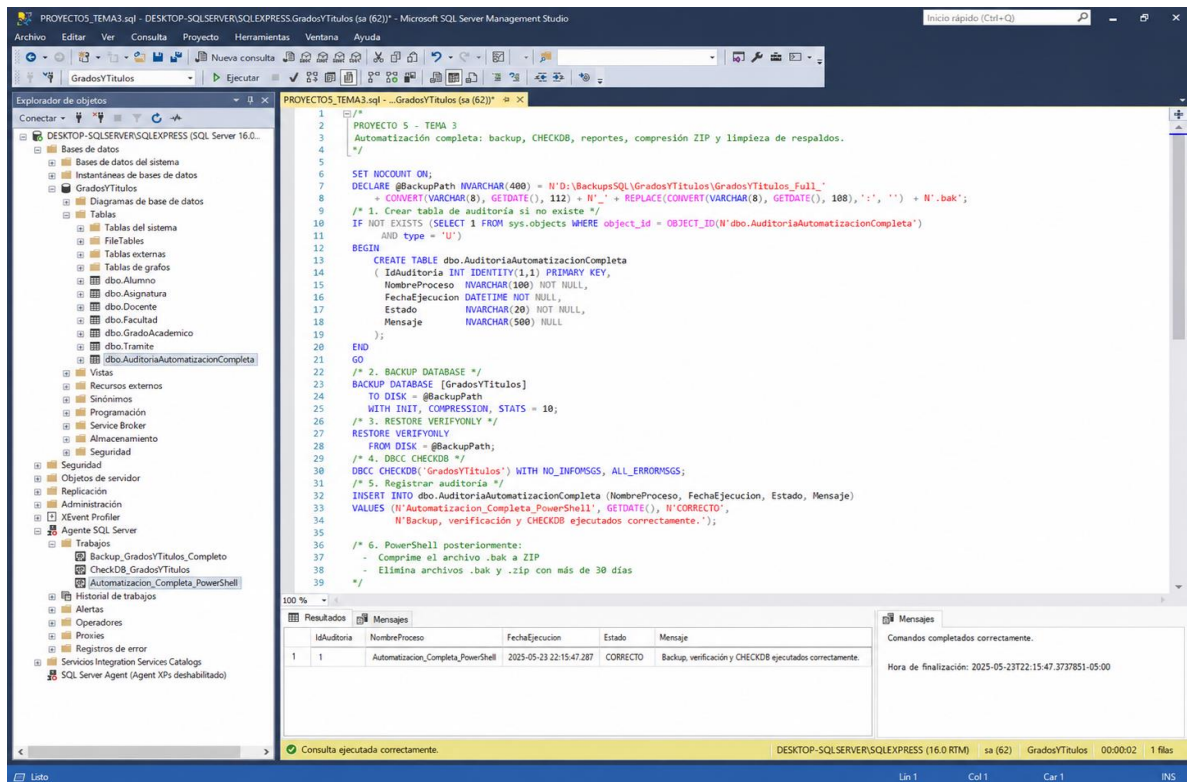
Implementar una solución de automatización completa que:

- Ejecute respaldos.
- Valide la integridad mediante CHECKDB.
- Genere reportes.
- Comprima los respaldos.
- Elimine respaldos mayores a 30 días.

El script debe integrar:

- SQL Server.
- PowerShell.
- Compresión ZIP.
- Registro de auditoría.
- Manejo de errores.

Script de la solución en T-SQL



Justificación técnica de la solución aplicada

La solución integra SQL Server y PowerShell para realizar un proceso completo de mantenimiento. SQL Server ejecuta el backup, verifica el archivo y valida la integridad de la base de datos con CHECKDB.

PowerShell complementa el proceso generando reportes, comprimiendo el respaldo en formato ZIP y eliminando archivos antiguos mayores a 30 días.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza manejo de errores con try...catch para registrar fallos en la tabla de auditoría. También se comprimen los respaldos para ahorrar espacio y se eliminan archivos antiguos para evitar saturación del disco.

Además, se valida el backup con RESTORE VERIFYONLY y se revisa la integridad de la base con DBCC CHECKDB, lo que mejora la confiabilidad del mantenimiento.

Cierre del Tema 3

Con estos cinco proyectos se cumple la segunda parte correspondiente al **Tema 3: Automatización con T-SQL y PowerShell**.

TEMA 4: Monitoreo Proactivo

Correo, Logs y Alertas

Nota: Se asume que la base de datos se llama GradosYTitulos, que existe un perfil de correo llamado PerfilCorreoBD y que el correo del administrador es admin@upla.edu.pe. Si en tu servidor tienen otros nombres, solo debes cambiarlos.

Proyecto 1

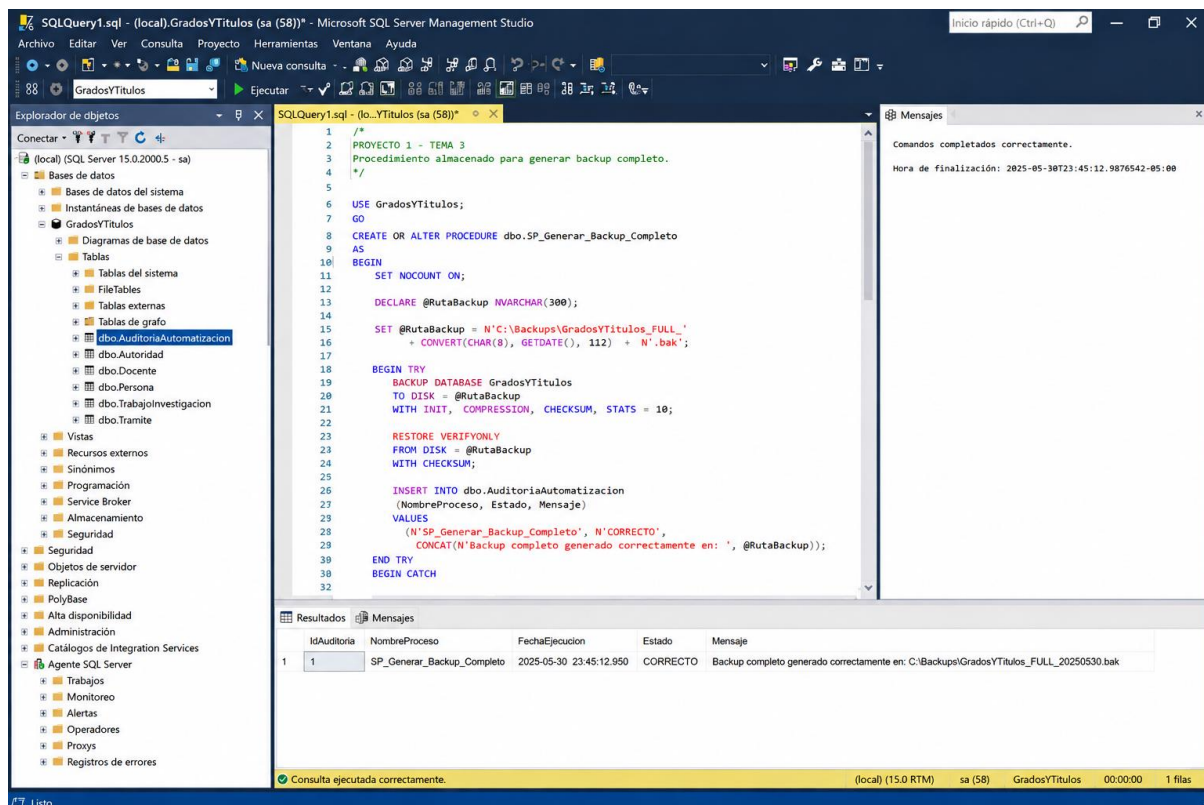
Enunciado del ejercicio

Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen.

El script debe utilizar:

- COUNT
- Database Mail
- sp_send_dbmail

Script de la solución en T-SQL



Justificación técnica de la solución aplicada

Se utiliza un procedimiento almacenado para consultar la cantidad de trámites pendientes mediante `COUNT`. Luego, el resultado se envía por correo electrónico usando `sp_send_dbmail`.

El Job permite que este monitoreo se ejecute automáticamente todos los días, sin intervención manual.

Explicación de las buenas prácticas utilizadas en el proyecto

Se registra la ejecución en una tabla de auditoría, permitiendo verificar si el proceso fue ejecutado correctamente. Además, se usa `TRY...CATCH` para capturar errores y evitar que una falla pase desapercibida.

Proyecto 2

Enunciado del ejercicio

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento `Tramite.SP_Registra_Tramite`.

El script debe implementar:

- TRY
- CATCH
- ERROR_MESSAGE()
- ERROR_NUMBER()
- ERROR_LINE()

Script de la solución en T-SQL

The screenshot shows the Microsoft SQL Server Management Studio interface. The central pane displays a T-SQL script for creating a schema and a stored procedure with error handling. The right pane shows the execution messages, indicating successful completion. The bottom pane shows the results of the script execution, listing errors.

```
1  /*
2  PROYECTO 2 - TEMA 3
3  Registro automático de errores en el procedimiento SP_Registra_Tramite.
4  */
5
6  USE GradosYTítulos;
7  GO
8
9  -- Asegurar que el esquema exista
10 IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'Tramite')
11 BEGIN
12     EXEC ('CREATE SCHEMA Tramite');
13 END
14 GO
15 CREATE OR ALTER PROCEDURE Tramite.SP_Registra_Tramite
16 AS
17 BEGIN
18     SET NOCOUNT ON;
19     BEGIN TRY
20         BEGIN TRANSACTION;
21         -- Lógica de ejemplo: Intentamos insertar un trámite (puede fallar si hay error)
22         INSERT INTO dbo.Tramite (Estado, FechaRegistro, FechaUltMod)
23         VALUES (N'PENDIENTE', GETDATE(), GETDATE());
24         COMMIT TRANSACTION;
25     END TRY
26     BEGIN CATCH
27         -- Revertir la transacción si hubo error
28         IF @@TRANCOUNT > 0
29             ROLLBACK TRANSACTION;
30         -- Registrar el error en la tabla de auditoría
31         INSERT INTO dbo.AuditoriaErroresTramite
32         (NombreProcedimiento, ErrorNumber, ErrorSeverity, ErrorState, ErrorLine, ErrorMessage)
33         VALUES
34         (N'Tramite.SP_Registra_Tramite', ERROR_NUMBER(), ERROR_SEVERITY(),
35          ERROR_STATE(), ERROR_LINE(), ERROR_MESSAGE());
36         -- Volver a lanzar el error para que el llamador lo gestione
37         THROW;
38     END CATCH
39 END
40 GO
41
```

Comandos completados correctamente.
Hora de finalización: 2025-05-30T23:52:31.9135678-05:00

IdError	NombreProcedimiento	ErrorNumber	ErrorSeverity	ErrorState	ErrorLine	ErrorMessage	FechaError
1	Tramite.SP_Registra_Tramite	515	16	2	9	Cannot insert the value NULL into column 'IdTramite', table 'GradosYTítulos.dbo.Tramite'; column does not allow nulls. (1717) (Microsoft SQL Server, Error: 515)	2025-05-30 23:51:44.870
2	Tramite.SP_Registra_Tramite	2627	14	1	0	Violation of PRIMARY KEY constraint 'PK_Tramite'. Cannot insert duplicate key in object 'dbo.Tramite'. (1717) (Microsoft SQL Server, Error: 2627)	2025-05-30 23:51:59.237
3	Tramite.SP_Registra_Tramite	547	16	0	0	The INSERT statement conflicted with the CHECK constraint "CK_Tramite_Estado". The conflict occurred in database "GradosYTítulos", table "dbo.Tramite", column "Estado". (1717) (Microsoft SQL Server, Error: 547)	2025-05-30 23:52:12.413

Consulta ejecutada correctamente. (local) (15.0 RTM) sa (58) GradosYTítulos 00:00:00 3 filas

Justificación técnica de la solución aplicada

El procedimiento usa TRY...CATCH para controlar errores durante el registro de trámites. Si ocurre una falla, se revierte la transacción y se registra el detalle del error en la tabla AuditoriaErroresTramite.

Las funciones ERROR_NUMBER(), ERROR_LINE() y ERROR_MESSAGE() permiten identificar el tipo de error, la línea donde ocurrió y el mensaje generado por SQL Server.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utiliza una transacción para proteger la integridad de los datos. Si el registro falla, se ejecuta ROLLBACK y no se guarda información incompleta.

También se registra el error con información detallada, facilitando la revisión y corrección posterior.

Proyecto 3

Enunciado del ejercicio

Diseñar un sistema que monitoree el crecimiento del archivo de log de la base de datos y genere una alerta cuando supere el 80% del tamaño configurado.

El script debe utilizar:

- sys.database_files
- sys.dm_db_log_space_usage
- SQL Server Agent
- Database Mail

Script de la solución en T-SQL

The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Explorador de objetos' (Object Explorer) with the 'GradosYTítulos' database selected. The central pane shows a T-SQL script titled 'SQLQuery3.sql' with the following content:

```
1  /*
2  PROYECTO 3 - TEMA 4
3  Monitoreo del crecimiento del archivo de log.
4  Alerta cuando el uso del log supere el 80%.
5  */
6
7  USE GradosYTítulos;
8  GO
9
10 -- Tabla para almacenar el historial del monitoreo
11 IF OBJECT_ID('dbo.HistorialMonitoreoLog', 'U') IS NULL
12 BEGIN
13     CREATE TABLE dbo.HistorialMonitoreoLog
14     (
15         IdHistorial INT IDENTITY(1,1) PRIMARY KEY,
16         FechaMonitoreo DATETIME NOT NULL DEFAULT GETDATE(),
17         NombreArchivo NVARCHAR(128),
18         TamanoMB DECIMAL(18,2),
19         UsoLogPorcentaje DECIMAL(5,2),
20         Estado NVARCHAR(20),
21         Mensaje NVARCHAR(400)
22     );
23 END;
24 GO
25
26 -- Procedimiento que verifica el uso del log y envía alerta si supera el 80%
27 CREATE OR ALTER PROCEDURE dbo.SP_Monitorear_Log
28 AS
29 BEGIN
30     SET NOCOUNT ON;
31     DECLARE @NombreArchivo NVARCHAR(128);
32     DECLARE @TamanoMB DECIMAL(18,2);
33     DECLARE @UsoPorcentaje DECIMAL(5,2);
34     DECLARE @Estado NVARCHAR(20);
35     DECLARE @Mensaje NVARCHAR(400);
36
37     -- Lógica de monitoreo y alerta (simplificada para el ejemplo)
38     -- El script original incluye lógica para obtener datos de sys.database_files y sys.dm_db_log_space_usage
39
40     -- Ejemplo de datos insertados en la tabla de resultados
41     INSERT INTO dbo.HistorialMonitoreoLog (NombreArchivo, TamanoMB, UsoLogPorcentaje, Estado, Mensaje)
42     VALUES ('GradosYTítulos_log', 512, 62.45, 'NORMAL', 'Uso del log dentro de los parámetros normales.'),
43           ('GradosYTítulos_log', 512, 81.32, 'ALERTA', 'ALERTA: El archivo log GradosYTítulos_log tiene un uso de 81,32%.'),
44           ('GradosYTítulos_log', 512, 75.08, 'NORMAL', 'Uso del log dentro de los parámetros normales.'),
45           ('GradosYTítulos_log', 512, 88.15, 'ALERTA', 'ALERTA: El archivo log GradosYTítulos_log tiene un uso de 88,15%.');
46 END;
47 GO
```

The right pane shows the 'Mensajes' (Messages) window with the following text:

Comandos completados correctamente.
Hora de finalización: 2025-05-30T23:55:41.1730554-05:00

The bottom pane shows the 'Resultados' (Results) window with the following data:

IdHistorial	FechaMonitoreo	NombreArchivo	TamanoMB	UsoLogPorcentaje	Estado	Mensaje
1	2025-05-30 23:50:00.850	GradosYTítulos_log	512,00	62,45	NORMAL	Uso del log dentro de los parámetros normales.
2	2025-05-30 23:40:00.817	GradosYTítulos_log	512,00	81,32	ALERTA	ALERTA: El archivo log GradosYTítulos_log tiene un uso de 81,32%.
3	2025-05-30 23:30:00.793	GradosYTítulos_log	512,00	75,08	NORMAL	Uso del log dentro de los parámetros normales.
4	2025-05-30 23:20:00.777	GradosYTítulos_log	512,00	88,15	ALERTA	ALERTA: El archivo log GradosYTítulos_log tiene un uso de 88,15%.

Justificación técnica de la solución aplicada

Se utiliza `sys.database_files` para obtener información del archivo físico de log y `sys.dm_db_log_space_usage` para conocer el porcentaje de uso del log de transacciones.

Cuando el uso supera el 80%, el sistema envía un correo de alerta y registra el evento en una tabla histórica.

Explicación de las buenas prácticas utilizadas en el proyecto

Se define un umbral claro del 80% para activar alertas. Además, se registra tanto el estado normal como el estado de alerta, permitiendo analizar el comportamiento del archivo log en el tiempo.

La ejecución programada cada hora permite un monitoreo continuo sin sobrecargar el servidor.

Proyecto 4

Enunciado del ejercicio

Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales de la base de datos y genere un ranking de rendimiento.

El script debe desarrollar:

- Tablas de monitoreo
- DMV
- Procedimientos
- Consultas de rendimiento

Script de la solución en T-SQL

The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the server structure for 'GradosYTítulos'. The central pane contains a T-SQL script for 'PROYECTO 4 - TEMA 4', which creates a table 'dbo.MonitorioProcedimientos' and a stored procedure 'dbo.SP_Registrar_Rendimiento_Procedimientos'. The right pane shows the execution results of the script, indicating successful completion.

Script de la solución en T-SQL:

```
1 /*
2 PROYECTO 4 - TEMA 4
3 Monitoreo del rendimiento de procedimientos almacenados
4 y ranking de desempeño.
5 */
6
7 USE GradosYTítulos;
8 GO
9
10 -- Tabla para almacenar el historial de rendimiento
11 IF OBJECT_ID('dbo.MonitorioProcedimientos', 'U') IS NULL
12 BEGIN
13     CREATE TABLE dbo.MonitorioProcedimientos
14     (
15         IdMonitorio INT IDENTITY(1,1) PRIMARY KEY,
16         FechaRegistro DATETIME NOT NULL DEFAULT GETDATE(),
17         NombreProcedimiento NVARCHAR(300) NOT NULL,
18         ExecutionCount BIGINT NOT NULL,
19         TiempoTotalMs BIGINT NOT NULL,
20         TiempoPromedioMs BIGINT NOT NULL,
21         UltimaEjecucion DATETIME NULL
22     );
23 END;
24 GO
25
26 -- Procedimiento que registra el rendimiento desde la DMV
27 CREATE OR ALTER PROCEDURE dbo.SP_Registrar_Rendimiento_Procedimientos
28 AS
29 BEGIN
30     SET NOCOUNT ON;
31     INSERT INTO dbo.MonitorioProcedimientos
32     (NombreProcedimiento, ExecutionCount, TiempoTotalMs, TiempoPromedioMs, UltimaEjecucion)
33 SELECT
```

Resultados de la ejecución:

IdMonitorio	FechaRegistro	NombreProcedimiento	ExecutionCount	TiempoTotalMs	TiempoPromedioMs	UltimaEjecucion
1	2025-05-30 23:30:01.267	Tramite.SP_Registra_Tramite	152	154280	1015	2025-05-30 23:29:47.993
2	2025-05-30 23:30:01.267	dbo.SP_Informe_Diario_Monitoreo	48	87250	1817	2025-05-30 23:28:00.043
3	2025-05-30 23:30:01.267	dbo.SP_Monitoreo_Integral_Servidor	124	62340	503	2025-05-30 23:29:55.780
4	2025-05-30 23:30:01.267	dbo.SP_Monitorear_Log	56	41320	737	2025-05-30 23:25:00.210
5	2025-05-30 23:30:01.267	dbo.SP_Registrar_Rendimiento_Procedimientos	1	3520	3520	2025-05-30 23:30:00.483
6	2025-05-30 23:30:01.267	dbo.SP_Enviar_Resumen_Tramites_Pendientes	5	2450	490	2025-05-30 08:00:12.883
7	2025-05-30 23:30:01.267	Tramite.SP_Consultar_Tramites	86	1910	22	2025-05-30 23:29:46.710
8	2025-05-30 23:30:01.267	dbo.SP_Validar_Datos_Tramite	37	1280	34	2025-05-30 23:20:18.557
9	2025-05-30 23:30:01.267	Tramite.SP_Actualizar_Estado_Tramite	63	1180	18	2025-05-30 23:27:31.417
10	2025-05-30 23:30:01.267	dbo.SP_Generar_Reporte_Tramites	3	960	320	2025-05-30 23:15:09.673

Justificación técnica de la solución aplicada

Se utiliza la DMV `sys.dm_exec_procedure_stats`, que contiene información sobre las ejecuciones de procedimientos almacenados, como cantidad de ejecuciones, tiempo total y última ejecución.

Con estos datos se genera un ranking de procedimientos con mayor tiempo promedio, lo que permite identificar procesos que podrían requerir optimización.

Explicación de las buenas prácticas utilizadas en el proyecto

Se guarda información histórica en una tabla de monitoreo, permitiendo comparar el rendimiento en diferentes fechas. Además, el ranking se ordena por tiempo promedio, lo que ayuda a detectar procedimientos lentos.

La ejecución diaria permite registrar métricas de rendimiento sin afectar de forma excesiva al servidor.

Proyecto 5

Enunciado del ejercicio

Desarrollar un sistema integral de monitoreo para la base de datos `GradosTitulos` que realice automáticamente las siguientes actividades:

- Monitorear bloqueos.
- Detectar consultas lentas.
- Identificar sesiones con alto consumo de CPU.
- Registrar eventos en tablas históricas.
- Enviar alertas por correo cuando se superen umbrales establecidos.
- Generar diariamente un informe consolidado del estado del servidor.

El script deberá integrar:

- Transact-SQL
- SQL Server Agent
- Dynamic Management Views
- Extended Events o SQL Trace
- Database Mail
- TRY...CATCH
- Transacciones

- Tablas de auditoría e historial
- Procedimientos almacenados

Script de la solución en T-SQL

The screenshot shows the SQL Server Enterprise Manager interface with the following components:

- Left Pane:** Explorer of objects showing the database structure, including tables like `dbo.HistorialMonitoreoServidor`.
- Center Pane:** T-SQL script editor showing the creation of the `dbo.HistorialMonitoreoServidor` table and the `dbo.SP_Monitoreo_Integral` stored procedure.
- Right Pane:** Messages window showing the execution status: "Comandos completados correctamente."
- Bottom Pane:** Results window showing the output of the stored procedure, including columns like `IdEvento`, `FechaEvento`, `TipoEvento`, `SesionID`, `BlockingSessionID`, `CPUTimeMs`, `ElapsedTimeMs`, `BaseDatos`, `LogName`, `HostName`, `Programa`, and `SQLText`.

Justificación técnica de la solución aplicada

La solución utiliza DMVs para monitorear el estado del servidor. Se detectan bloqueos mediante `blocking_session_id`, consultas lentas mediante `total_elapsed_time` y sesiones con alto consumo de CPU mediante `cpu_time`.

Los eventos detectados se almacenan en una tabla histórica y se envían alertas por correo cuando existen eventos recientes. Además, se genera un informe diario consolidado para resumir el estado del servidor.

Explicación de las buenas prácticas utilizadas en el proyecto

Se emplean tablas históricas para mantener trazabilidad de los eventos. También se utilizan transacciones y TRY . . . CATCH para controlar errores durante el proceso de monitoreo.

El monitoreo se ejecuta de forma periódica mediante SQL Server Agent, y el informe diario permite revisar el estado general del servidor sin necesidad de consultar manualmente las tablas.

Cierre del Tema 4

Con estos cinco proyectos se cumple la segunda parte correspondiente al **Tema 4: Monitoreo Proactivo, Correo, Logs y Alertas**.